As an Executive Director at Nomura, I consider the integrity of our backtesting framework to be the most critical technical barrier between a profitable strategy and a "garbage-in, garbage-out" disaster. In the Central Risk Book (CRB), we are often predicting volatility or alpha decay over horizons that overlap—for example, a 5-day realized volatility forecast.

If you use standard k-fold cross-validation, your training sets will contain observations that share the same return data as the test sets, creating **information leakage**. This leads to a massive overestimation of performance that disappears the moment you deploy capital. Here is the technical deep-dive into how we solve this using Combinatorial Purged Cross-Validation (CPCV).

---

1. The Mathematical Deep Dive: Managing Temporal Contamination

In a financial time series, the fundamental assumption of k-fold—that observations are independent and identically distributed (i.i.d.)—is violated by temporal correlation and overlapping labels.

Step 1: The Leakage Problem

If we predict a target Y (e.g., 5-day volatility) using features X, and our test fold includes time t, any training observation t' where $|t - t'| < h$ (where h is the prediction horizon) will share partial information with the test target. Using these t' points to train the model is **look-ahead bias**, as the training set essentially "sees" a portion of the future test labels.

Step 2: The Purging Logic

To resolve this, we identify the set of indices $\mathcal{T}_{test}$ belonging to the test fold. For every index $\tau \in \mathcal{T}_{test}$, we must exclude any training index τ' such that:

$$\tau' \in [\tau - h + 1, \tau + h - 1]$$

This is **Purging**. We are surgically removing training samples that are causally or informationally contaminated by the test period.

Step 3: The Embargo Mechanism

Even after purging, financial data often exhibits serial correlation (e.g., a shock at time t persists into $t+1$). Simple purging only removes the overlap; it does not protect against the predictive power of recent market events leaking into the training set via auto-correlation. We define an **Embargo** period of length G (typically a fraction of the horizon h) immediately following the test set. Any samples in the training set that occur during this window are discarded, preventing the model from learning from information that was effectively "just released" relative to the test period.

Step 4: Combinatorial Paths

Standard Walk-Forward CV produces only one path (e.g., train 1..100, test 101..120). This is a single, fragile realization of the market. **CPCV** treats the K folds as independent blocks and trains on all possible combinations of these blocks. If we have K folds, we generate:

$$\binom{K}{K-2}$$

unique backtest paths. This gives us a **distribution of performance** rather than a single point estimate, allowing us to quantify the statistical significance of the strategy's Sharpe ratio and its sensitivity to different market regimes.

---

2. Application of Theory and Production Code

To implement this at Nomura, we need a robust, vectorized CPCV generator that can handle large-scale equity data and perform the purging/embargoing steps efficiently without redundant loops.

Production-Grade Python CPCV Implementation

```
import numpy as np
import pandas as pd
from itertools import combinations

class CPCVEngine:
    """
    Production-grade Combinatorial Purged Cross-Validation for CRB risk models.
    Prevents look-ahead bias and generates multiple backtest paths.
    """
    def __init__(self, n_folds=10, purge_gap=5):
        self.n_folds = n_folds
        self.purge_gap = purge_gap

    def get_train_test_split(self, n_samples: int):
        """
        Generates purged training and testing indices for all combinatorial paths.
        """
        # 1. Define fold boundaries
        indices = np.arange(n_samples)
        fold_size = n_samples // self.n_folds
        fold_bounds = [(i * fold_size, (i + 1) * fold_size) for i in
range(self.n_folds)]

        # 2. CPCV: Select K-2 folds for training, 2 for testing (typical path)
        for test_indices in combinations(range(self.n_folds), 2):
            test_mask = np.zeros(n_samples, dtype=bool)
            purge_mask = np.zeros(n_samples, dtype=bool)

            # Identify test set indices
            for f in test_indices:
                start, end = fold_bounds[f]
                test_mask[start:end] = True

                # Apply Purging: Remove data surrounding test folds
                purge_start = max(0, start - self.purge_gap)
                purge_end = min(n_samples, end + self.purge_gap)
                purge_mask[purge_start:purge_end] = True

            # Training set is everything NOT in the test or purge masks
            train_mask = ~(test_mask | purge_mask)
```

```
            yield indices[train_mask], indices[test_mask]

# Example Usage
# engine = CPCVEngine(n_folds=10, purge_gap=5)
# for train_idx, test_idx in engine.get_train_test_split(1000):
#     model.fit(X.iloc[train_idx], y.iloc[train_idx])
#     preds = model.predict(X.iloc[test_idx])
```

---

3. Executive Director's Follow-up Questions

If you presented this approach, I would immediately challenge the statistical assumptions embedded in the combinatorial paths.

Follow-Up 1: "CPCV generates paths that are overlapping in their training data. Therefore, the resulting backtest paths are not independent. How do you account for this when calculating the variance of your model's performance?"

- **The Answer:** You are absolutely right. The paths are highly correlated because they share significant portions of the training data. Taking a simple average of the Sharpe ratios across all paths will yield an overly optimistic standard error.
- **The Fix:** We must calculate the **Effective Number of Independent Paths**. We use the **Combinatorial covariance matrix** to weight the paths. Specifically, we estimate the covariance matrix of the model errors across all paths and use this to compute a "de-biased" performance variance. In practice, we look at the **Information Coefficient (IC) decay** across these paths to determine if the model's predictive power is robust or if it was driven by specific "lucky" training folds.

Follow-Up 2: "In the context of the Central Risk Book (CRB), we deal with multi-asset equity correlation. If I purge and embargo based on a single asset's timeframe, am I still susceptible to cross-asset leakage?"

- **The Answer:** Yes. If we are modeling assets that move together (e.g., high-beta equities in the same sector), a purge on Asset A is insufficient if Asset B carries predictive information about Asset A during the test window.
- **The Fix:** We must implement **Cross-Asset Purging**. When we train the model, we purge observations for *all* assets that have a correlation threshold $|\rho| > 0.5$ with the assets in the test fold, even if they are ostensibly different symbols. We maintain a dynamic covariance matrix of the universe and perform global purging for any date in the training set that occurs within the h-day horizon of a test date for any correlated asset.